

Implementacja ORM w C#

Dokumentacja Architektury i Mechanizmów Mapowania

Kevin Stuka, Jarosław Klima

Szymon Kowalski, Daniel Sterzel

Design Patterns

2025

Contents

1	Cel projektu	4
2	Zakres funkcjonalności	4
2.1	Mapowanie i Metadane (Mapping Layer)	4
2.2	Zarządzanie Stanem i Transakcyjność (Core Layer)	5
2.3	Przetwarzanie Zapytań (Query Layer)	6
2.4	Dostęp do Danych (SQL Implementation Layer)	6
3	Wykorzystane technologie	6
4	Konteneryzacja projektu przy użyciu Dockera	7
4.1	Plik Dockerfile	7
4.2	Uruchamianie projektu	8
4.3	Zalety zastosowania Dockera	8
5	Architektura Fizyczna	9
6	Architektura Logiczna	9
6.1	Warstwa API (Client Interface)	10
6.2	Warstwa Jadra (Core Runtime)	10
6.3	Warstwa Mapowania i Metadanych	11
6.4	Warstwa Przetwarzania Zapytań (Query Engine)	11
6.5	Warstwa Dostępu do Danych i Generowania SQL	11
7	System Atrybutów (Konfiguracja)	12

8	Architektura Budowania Modelu	12
8.1	ModelDirector (Dyrektor)	13
8.2	ReflectionModelBuilder (Konkretny Budowniczy)	13
8.3	Strategie Nazewnictwa	13
8.4	Algorytm rozwiązywania klucza głównego	13
9	Reprezentacja Metadanych (Metadata Object)	14
9.1	EntityMap – Mapa Tabeli i Dziedziczenia	14
9.2	PropertyMap – Mapa Właściwości	14
10	Magazyn Metadanych (MetadataStore)	14
10.1	Registry i Wstrzykiwanie Zależności	14
10.2	Konfiguracja przez MetadataStoreBuilder	15
11	Rola DbContext	16
11.1	Wzorzec Unit of Work	16
11.2	Wzorzec Identity Map (First-Level Cache)	16
12	Śledzenie Zmian (ChangeTracker)	17
12.1	Stany encji (EntityState)	17
12.2	Proces Zapisu (SaveChanges)	17
13	Fasada Dostępu (DbSet)	18
14	Architektura Providera LINQ	19
15	Translacja Drzew Wyrażeń (SqlExpressionVisitor)	19
15.1	Obsługa metod LINQ	19
15.2	Translacja Operatorów	20
16	Bezpieczeństwo i Parametryzacja	21
17	Model Pośredni (QueryModel)	21
18	Abstrakcja Generatora (ISqlGenerator)	22
19	Implementacja dla SQLite (SqliteSqlGenerator)	22
19.1	Obsługa Autoinkrementacji (Identity)	23
19.2	Cytowanie i Parametry	23
20	Budowanie Zapytań (SqlQueryBuilder)	23
21	Rola ObjectMaterializer	25

22 Obsługa Polimorfizmu (Inheritance Support)	25
23 Optymalizacja z użyciem Expression Trees	26
23.1 Szybka Fabryka Obiektów	26
23.2 Kompilowane Settery	26
24 Bezstanowość i Indeksy Kolumn (Ordinals)	26
25 Obsługa Typów i Bezpieczeństwo	27
26 Wzorzec Builder (Budowniczy)	29
27 Wzorzec Unit of Work (Jednostka Pracy)	31
28 Wzorzec Identity Map (Mapa Tożsamości)	34
29 Obsługa Dziedziczenia (Inheritance Mapping)	37
29.1 Obsługiwane Strategie	37
29.2 Algorytm Doboru Strategii (Heurystyka Buildera)	38
29.3 Walidacja Spójności Modelu	39
30 Diagram Sekwencji: Inicjalizacja Modelu	40
31 Diagram Sekwencji: Wykonanie Zapytania LINQ	40
32 Diagram Aktywności (Activity Diagram)	41
33 Osiągnięte cele	43
34 Napotkane wyzwania	43
35 Plany rozwoju	43

1 Cel projektu

Głównym celem niniejszego projektu było zaprojektowanie i implementacja własnego, lekkiego systemu mapowania obiektowo-relacyjnego (ORM – Object-Relational Mapping) w środowisku .NET. Projekt ten powstał jako odpowiedź na potrzeby zrozumienia niskopoziomowych mechanizmów zarządzających dostępem do danych oraz jako praktyczne studium zastosowania zaawansowanych wzorców projektowych (GoF) w realnym systemie informatycznym.

System ma za zadanie rozwiązać problem *impedance mismatch* – niedopasowania między obiektywnym paradygmatem języka C# a relacyjną strukturą bazy danych SQLite. Kluczowym założeniem projektowym była rezygnacja z gotowych rozwiązań (takich jak Entity Framework Core czy Dapper) na rzecz autorskiej implementacji, która zapewnia pełną kontrolę nad procesem translacji zapytań, zarządzania stanem obiektów oraz generowania kodu SQL.

Szczególny nacisk położono na trzy aspekty jakościowe:

- **Wydajność:** Minimalizacja narzutu (overhead) poprzez unikanie refleksji w czasie działania aplikacji (runtime) na rzecz kompilacji *Expression Trees* oraz cache'owania metadanych.
- **Modularność:** Zastosowanie zasad SOLID (w szczególności zasady otwarte-zamknięte i pojedynczej odpowiedzialności) pozwoliło na odseparowanie warstwy definicji modelu, warstwy generowania zapytań oraz warstwy wykonawczej.
- **Testowalność:** Architektura oparta na wstrzykiwaniu zależności (Dependency Injection) i interfejsach umożliwia łatwe testowanie jednostkowe poszczególnych komponentów bez konieczności połączenia z fizyczną bazą danych.

2 Zakres funkcjonalności

Zrealizowany system ORM obejmuje pełen cykl życia dostępu do danych, od konfiguracji modelu po materializację wyników. Funkcjonalność systemu można podzielić na następujące obszary:

2.1 Mapowanie i Metadane (Mapping Layer)

System implementuje podejście *Code First* z silnym naciskiem na automatyzację poprzez *Convention over Configuration*, jednocześnie oferując pełne wsparcie dla paradygmatów obiektowych.

- **Inteligentne budowanie modelu:** Wykorzystując wzorzec *Builder* i *Director*, system nie tylko skanuje zestawy (Assembly), ale także przeprowadza topologiczne uporządkowanie hierarchii dziedziczenia. Pozwala to na poprawne budowanie map dla skomplikowanych hierarchii klas (rodzic-dziecko).
- **Mapowanie Dziedziczenia (Polimorfizm):** System automatycznie rozpoznaje relacje dziedziczenia i mapuje je na strukturę relacyjną przy użyciu trzech strategii:
 - *Table Per Hierarchy (TPH)*: Domyślna strategia mapująca całą hierarchię klas do jednej tabeli z automatycznie zarządzaną kolumną dyskryminatora.
 - *Table Per Concrete Class (TPC)*: Strategia tworząca osobne tabele dla każdej klasy konkretnej, aktywowana poprzez jawne użycie atrybutu `[Table]` na klasie pochodnej.
 - *Table Per Type (TPT)*: Strategia tworząca oddzielne tabele dla każdej klasy w hierarchii. Tabela klasy pochodnej zawiera wyłącznie jej własne pola i jest powiązana kluczem obcym z tabelą klasy bazowej.
- **Konfiguracja atrybutowa:** Możliwość precyzyjnego nadpisywania domyślnych konwencji za pomocą atrybutów deklaratywnych: `[Table]`, `[Key]`, `[Ignore]`. Atrybuty są analizowane z uwzględnieniem kontekstu dziedziczenia.
- **Zoptymalizowany Magazyn Metadanych:** Centralny, niemutowalny `MetadataStore` przechowujący definicje `EntityMap`. Struktura ta jest budowana raz (przy starcie) i zapewnia dostęp do metadanych w czasie $O(1)$, eliminując narzut Refleksji podczas przetwarzania zapytań (Runtime).

2.2 Zarządzanie Stanem i Transakcyjność (Core Layer)

Jadrem systemu jest implementacja wzorca *Unit of Work* oraz *Identity Map*.

- **DbContext:** Główny punkt wejścia dla aplikacji, zarządzający połączeniem z bazą danych i cyklem życia transakcji.
- **Change Tracking:** Mechanizm śledzenia zmian (`ChangeTracker`), który monitoruje stan obiektów (*Added*, *Modified*, *Deleted*, *Unchanged*, *Detached*). Pozwala to na zapis wszystkich zmian do bazy danych w jednej transakcji za pomocą metody `SaveChanges()`.
- **First-Level Cache:** Implementacja pamięci podręcznej w obrębie kontekstu, zapobiegająca wielokrotnemu pobieraniu tej samej encji z bazy danych w ramach jednego zadanania.

2.3 Przetwarzanie Zapytań (Query Layer)

System udostępnia interfejs zgodny z LINQ (*Language Integrated Query*), co pozwala programistom na pisanie zapytań w języku C#.

- **LINQ Provider:** Implementacja interfejsów `IQueryProvider` i `IQueryable`, umożliwiająca budowanie zapytań w sposób leniwy (*Deferred Execution*).
- **Translacja Wyrażeń:** Wykorzystanie wzorca *Visitor* (`SqlExpressionVisitor`) do translacji Drzew Wyrażeń (*Expression Trees*) na struktury reprezentujące zapytania SQL. Obsługiwane są operacje filtrowania (`Where`), sortowania (`OrderBy`), oraz paginacji (`Take`, `Skip`).

2.4 Dostęp do Danych (SQL Implementation Layer)

Niskopoziomowa warstwa odpowiedzialna za fizyczną komunikację z bazą danych.

- **Abstrakcja Generatora:** Interfejs `ISqlGenerator` umożliwiający łatwą wymianę dialektu SQL (np. migracje z SQLite na PostgreSQL).
- **Fluent SQL Builder:** Narzędzie `SqlQueryBuilder` do bezpiecznego i czytelnego konstruowania łańcuchów znaków SQL, chroniące przed błędami składniowymi.
- **Parametryzacja:** Pełna obsługa parametrów zapytań w celu ochrony przed atakami typu *SQL Injection*.
- **Materializacja:** Wysoce wydajny `ObjectMaterializer`, który wykorzystuje kompilowane w locie delegaty do przekształcania surowych danych `IDataRecord` w silnie typowane obiekty .NET.

3 Wykorzystane technologie

Projekt został zrealizowany w oparciu o nowoczesny ekosystem platformy .NET, wykorzystując następujące technologie i narzędzia:

- **Platforma:** .NET 9 – najnowsza wersja frameworka firmy Microsoft, zapewniająca wysoką wydajność oraz dostęp do nowoczesnych funkcji języka C# (np. `record types`, `nullable reference types`, usprawnienia w `System.Linq.Expressions`).
- **Baza Danych:** SQLite (z wykorzystaniem biblioteki `Microsoft.Data.Sqlite`) – wybrana ze względu na swoją lekkość, brak konieczności instalacji serwera oraz łatwość w przenoszeniu (plikowa baza danych), co jest idealne dla celów demonstracyjnych i edukacyjnych.

- **Testy:** xUnit – framework do testów jednostkowych i integracyjnych, wykorzystany do weryfikacji poprawności mapowania, generowania SQL oraz materializacji obiektów.
- **Refleksja i Metaprogramowanie:** Intensywne wykorzystanie przestrzeni nazw `System.Reflection` (do skanowania metadanych przy starcie) oraz `System.Linq.Expressions` (do kompilacji kodu w czasie rzeczywistym dla zapewnienia wydajności zbliżonej do kodu natywnego).

4 Konteneryzacja projektu przy użyciu Dockera

W celu zapewnienia powtarzalności, przenośności oraz niezależności od lokalnego środowiska deweloperskiego, projekt został skonteneryzowany przy użyciu narzędzia **Docker**. Kontener Dockera dostarcza kompletne środowisko uruchomieniowe zawierające .NET SDK, wszystkie zależności oraz kod źródłowy projektu.

Projekt jest rozwiązaniem opartym o platforme .NET 9 i składa się z:

- biblioteki ORM,
- projektu testów jednostkowych,
- oraz prostej aplikacji testowej.

Celem konteneryzacji jest umożliwienie budowania i testowania projektu na dowolnym komputerze bez konieczności instalowania środowiska .NET lub dodatkowych narzędzi.

4.1 Plik Dockerfile

Środowisko kontenera jest definiowane przez plik `Dockerfile` umieszczony w katalogu głównym rozwiązania. Wykorzystuje on oficjalny obraz Microsoft .NET SDK i realizuje następujące kroki:

- kopiuje kod źródłowy do kontenera,
- przywraca zależności NuGet,
- buduje projekt testowy,
- a następnie uruchamia wszystkie testy jednostkowe.

```
FROM mcr.microsoft.com/dotnet/sdk:9.0-preview
```

```
WORKDIR /app
```

```
COPY . .
```

```
RUN dotnet restore
```

```
RUN dotnet build ORM.Tests/ORM.Tests.csproj -c Debug
```

```
CMD ["dotnet", "test", "ORM.Tests/ORM.Tests.csproj",  
    "-c", "Debug", "--no-build", "--verbosity", "normal"]
```

Taka konfiguracja zapewnia deterministyczny i powtarzalny proces budowania oraz testowania projektu.

4.2 Uruchamianie projektu

Po sklonowaniu repozytorium projekt można zbudować i uruchomić przy użyciu następujących poleceń:

```
docker build -t orm .  
docker run orm
```

Pierwsze polecenie buduje obraz Dockera, pobierając wymagane środowisko .NET oraz wszystkie zależności. Drugie polecenie uruchamia kontener, który automatycznie kompiluje projekt i wykonuje testy jednostkowe.

Jeżeli wykonanie zakończy się bez błędów, oznacza to poprawne zbudowanie i weryfikację projektu.

4.3 Zalety zastosowania Dockera

Zastosowanie Dockera zapewnia następujące korzyści:

- identyczne działanie projektu na każdym systemie operacyjnym,
- brak konieczności instalacji środowiska .NET,
- izolacje i wersjonowanie zależności,
- pełna powtarzalność procesu budowania i testowania.

Dzięki temu projekt jest gotowy do automatycznego testowania, integracji ciągłej oraz oceny przez osoby trzecie.

Architektura Systemu

Architektura systemu ORM została zaprojektowana w sposób modułowy, z wyraźnym podziałem na odpowiedzialności (Separation of Concerns). Pozwala to na niezależny rozwój poszczególnych komponentów, łatwe testowanie oraz wymianę implementacji (np. generatora SQL) bez wpływu na pozostałe części systemu. Poniżej przedstawiono architekturę w ujęciu fizycznym (organizacja kodu) oraz logicznym (warstwy funkcjonalne).

5 Architektura Fizyczna

Architektura fizyczna projektu odzwierciedla organizację plików źródłowych, przestrzeni nazw (namespaces) oraz fizycznych zależności między komponentami wewnątrz solution .NET.

Struktura projektu została podzielona na odseparowane katalogi pełniące role modułów, co zapobiega powstawaniu cyklicznych zależności i wymusza czystość architektury.

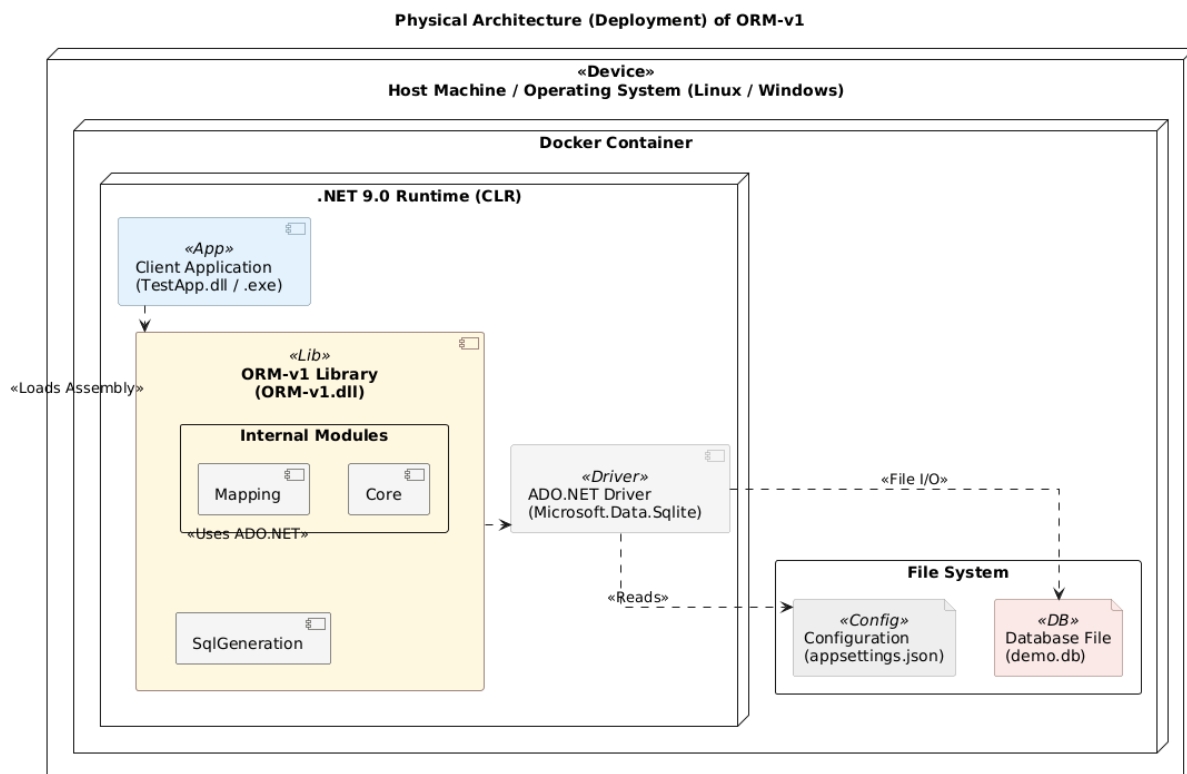


Figure 1: Diagram architektury fizycznej i zależności między modułami systemu ORM.

6 Architektura Logiczna

Architektura logiczna przedstawia system jako zestaw współpracujących ze sobą warstw, przez które przepływają dane od momentu wywołania metody przez programistę, aż do

wykonania zapytania na bazie danych.

Zastosowano tutaj klasyczny model warstwowy, gdzie warstwy wyższe korzystają z usług warstw niższych, co zapewnia przejrzystość przepływu sterowania.

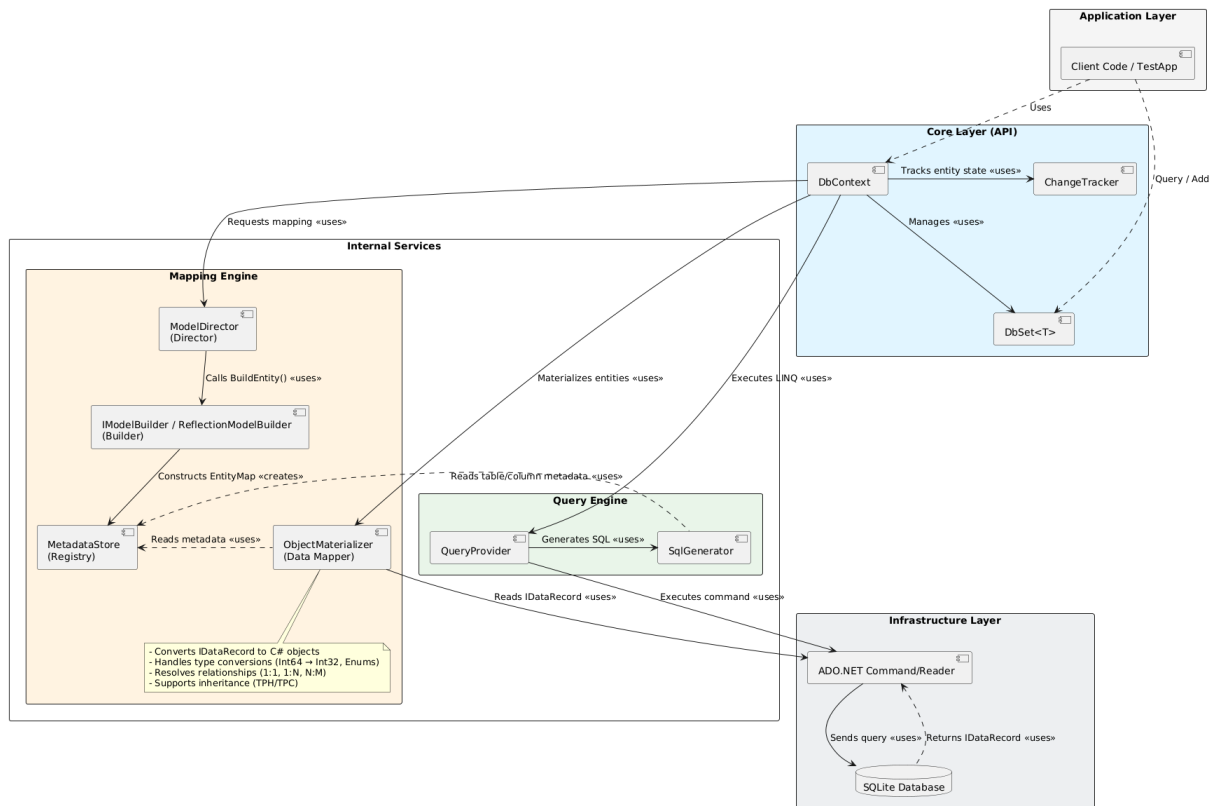


Figure 2: Diagram architektury logicznej przedstawiający warstwy systemu i przepływ danych.

Wyróżniono pięć głównych warstw logicznych:

6.1 Warstwa API (Client Interface)

Jest to publiczny interfejs biblioteki, z którym bezpośrednio wchodzi w interakcje programista aplikacji klienckiej.

- **Komponenty:** `DbContext`, `DbSet<T>`, `DatabaseFacade`.
- **Odpowiedzialność:** Udostępnianie metod do wykonywania operacji CRUD (`Add`, `Remove`), budowania zapytań (`Where`, `Select`) oraz zarządzania transakcjami (`SaveChanges`). Warstwa ta ukrywa skomplikowaną logikę wewnętrzną przed użytkownikiem.

6.2 Warstwa Jadra (Core Runtime)

Serce systemu, odpowiedzialne za zarządzanie stanem i koordynację prac.

- **Komponenty:** `ChangeTracker`, `EntityEntry`, `UnitOfWork`.
- **Odpowiedzialność:** Śledzenie zmian w obiektach (wykrywanie stanów *Added*, *Modified*, *Deleted*), implementacja wzorca *Identity Map* (zapewnienie unikalności instancji w pamięci) oraz orkiestracja procesu zapisu zmian do bazy danych.

6.3 Warstwa Mapowania i Metadanych

Warstwa dostarczająca wiedzę o strukturze danych („Mózg” systemu).

- **Kluczowe Komponenty:** `MetadataStore` (Registry), `EntityMap` (Product), `ModelDirector` (Director) oraz `ReflectionModelBuilder` (Concrete Builder).
- **Odpowiedzialność:** Definiowanie reguł transformacji obiektów C# na relacyjny model SQL. Warstwa ta wykorzystuje wzorzec *Builder* do analizy hierarchii klas i strategii dziedziczenia (TPH/TPC). Jest inicjalizowana jednokrotnie przy starcie, udostępniając zoptymalizowane struktury danych (słowniki $O(1)$) dla silnika zapytań i materializera.

6.4 Warstwa Przetwarzania Zapytań (Query Engine)

Odpowiada za tłumaczenie języka LINQ na reprezentacje pośrednia.

- **Komponenty:** `SqlExpressionVisitor`, `QueryModel`, `SqlQueryBuilder`.
- **Odpowiedzialność:** Analiza Drzew Wyrażeń (Expression Trees) dostarczonych przez kompilator C#, walidacja poprawności zapytania względem modelu oraz budowa modelu pośredniego (`QueryModel`), który jest niezależny od konkretnego dialektu SQL.

6.5 Warstwa Dostępu do Danych i Generowania SQL

Warstwa najniższa, odpowiedzialna za fizyczną komunikację.

- **Komponenty:** `ISqlGenerator`, `SqliteSqlGenerator`, `IDbCommand`, `ObjectMaterializer`.
- **Odpowiedzialność:** Zamiana modelu pośredniego na tekst zapytania SQL specyficzny dla bazy SQLite, wykonanie polecenia poprzez ADO.NET oraz materializacja surowych wyników (`IDataReader`) z powrotem na obiekty domenowe.

Warstwa Mapowania i Metadanych

Warstwa mapowania (*Mapping Layer*) stanowi fundament całego systemu ORM. Jej głównym zadaniem jest przetłumaczenie struktury klas obiektowych (C#) na relacyjny model bazy danych oraz dostarczenie zoptymalizowanych metadanych dla pozostałych modułów systemu.

W przeciwieństwie do rozwiązań opartych na ciągłej refleksji (runtime reflection), niniejszy projekt realizuje podejście polegające na jednokrotnym zbudowaniu modelu w momencie startu aplikacji. Takie rozwiązanie gwarantuje, że kosztowna analiza typów nie wpływa na wydajność wykonywania zapytań.

7 System Atrybutów (Konfiguracja)

System wykorzystuje podejście *Configuration by Exception* (Konfiguracja przez wyjątek). Oznacza to, że ORM domyślnie mapuje nazwy klas i właściwości bezpośrednio na nazwy tabel i kolumn (z uwzględnieniem przyjętej konwencji nazewnictwa). Atrybuty są wymagane tylko wtedy, gdy chcemy odstępować od tych domyślnych reguł.

Zaimplementowane atrybuty (znajdujące się w przestrzeni nazw `ORM_v1.Attributes`) to:

- `[Table("nazwa")]` – Nakładany na klasę. Wskazuje jawną nazwę tabeli w bazie danych. Jest to przydatne, gdy nazwa klasy (np. `User`) różni się od nazwy tabeli (np. `tbl_sys_users`).
- `[Column("nazwa")]` – Nakładany na właściwość. Mapuje właściwość na konkretną nazwę kolumny. Posiada najwyższy priorytet, nadpisując wszelkie strategię nazewnictwa.
- `[Key]` – Oznacza właściwość będącą kluczem głównym (*Primary Key*) encji. Jest kluczowy dla mechanizmu `ChangeTracker` do identyfikacji unikalności obiektów.
- `[Ignore]` – Wyklucza właściwość z mapowania. Pola oznaczone tym atrybutem nie będą uwzględniane w zapytaniach SQL (INSERT/SELECT). Stosowany np. dla właściwości obliczanych tylko w pamięci.
- `[ForeignKey("NazwaKlucza")]` – Wskazuje, która właściwość skalarna (np. `AuthorId`) jest kluczem obcym dla danej właściwości nawigacyjnej (np. `Author`). Umożliwia to poprawne budowanie relacji w przyszłych rozszerzeniach ORM (np. JOIN).

8 Architektura Budowania Modelu

Proces mapowania klas na strukturę bazy danych został zrefaktoryzowany przy użyciu klasycznego wzorca projektowego *Builder*. Odpowiedzialność została rozdzielona na dwa kluczowe komponenty:

8.1 ModelDirector (Dyrektor)

Klasa `ModelDirector` zarządza procesem i kolejnością budowania, nie wnikając w szczegóły techniczne samej konstrukcji mapy. Odpowiada za:

1. **Skanowanie Assembly:** Przeszukiwanie zestawu danych w celu znalezienia kandydatów na encje.
2. **Sortowanie Topologiczne:** Ustalenie kolejności budowania od klas bazowych do pochodnych (Rodzic $\rightarrow$ Dziecko), co jest krytyczne dla poprawnego mapowania dziedziczenia.
3. **Heurystyka wykrywania encji:** Klasa jest uznawana za encje, jeśli posiada atrybut `[Table]` lub właściwość klucza głównego (`[Key]`, `Id`).

8.2 ReflectionModelBuilder (Konkretny Budowniczy)

Klasa implementująca interfejs `IModelBuilder`, odpowiedzialna za techniczne aspekty tworzenia obiektu `EntityMap` dla pojedynczego typu. Jej zadania to:

- Analiza atrybutów przy użyciu Refleksji.
- Dobór strategii dziedziczenia (TPH/TPC).
- Rozwiązywanie nazw tabel i kolumn przy użyciu strategii nazewnictwa.

8.3 Strategie Nazewnictwa

Projekt implementuje wzorzec *Strategy* dla konwencji nazewnictwa (`INamingStrategy`). Dostępne implementacje:

- **PascalCaseNamingStrategy:** Tożsamościowa (`FirstName` $\rightarrow$ `FirstName`).
- **SnakeCaseNamingStrategy:** Konwersja SQL (`FirstName` $\rightarrow$ `first_name`).

8.4 Algorytm rozwiązywania klucza głównego

`ReflectionModelBuilder` poszukuje klucza w kolejności:

1. Właściwość z atrybutem `[Key]`.
2. Właściwość o nazwie `Id`.
3. Właściwość o nazwie `{TypeName}Id`.

9 Reprezentacja Metadanych (Metadata Object)

Wiedza o strukturze bazy danych jest przechowywana w zoptymalizowanych strukturach `EntityMap`.

9.1 EntityMap – Mapa Tabeli i Dziedziczenia

Obiekt `EntityMap` jest niemutowalny (*Immutable*). W najnowszej wersji systemu został rozbudowany o obsługę polimorfizmu. Jego kluczowe pola to:

- **InheritanceStrategy:** Określa strategię mapowania hierarchii:
 - `TablePerHierarchy` (TPH): Cała hierarchia w jednej tabeli z kolumną dyskryminatora.
 - `TablePerConcreteClass` (TPC): Każda klasa posiada własną, niezależną tabelę.
- **Discriminator:** Wartość tekstowa (np. "Dog") identyfikująca typ w bazie danych (używane przy TPH).
- **BaseMap:** Referencja do mapy klasy bazowej. Pozwala generatorowi SQL na budowanie zapytań uwzględniających pola odziedziczone.
- **Podział właściwości:**
 - `ScalarProperties`: Typy proste mapowane na kolumny.
 - `NavigationProperties`: Relacje do innych encji.
- **Lookup Cache ($O(1)$):** Słowniki pozwalające na błyskawiczne wyszukiwanie mapowania kolumn.

9.2 PropertyMap – Mapa Właściwości

Opisuje pojedynczą kolumnę lub relację. Obsługuje typy `Nullable` oraz posiada flagę `IsNavigation`, która zapobiega mapowaniu obiektów złożonych jako prostych kolumn SQL.

10 Magazyn Metadanych (MetadataStore)

Centralny kontener przechowujący wszystkie mapy.

10.1 Registry i Wstrzykiwanie Zależności

`MetadataStore` pełni rolę rejestru (*Registry Pattern*). Nie jest statycznym singletonem, lecz instancja wstrzykiwana do `DbContext`, co wspiera testowalność i izolację.

10.2 Konfiguracja przez MetadataStoreBuilder

Ze względu na złożoność procesu (użycie Dyrektora i Budowniczego), inicjalizacja odbywa się przez `MetadataStoreBuilder` z interfejsem *Fluent API*:

```
var metadata = new MetadataStoreBuilder()
    .AddAssembly(typeof(User).Assembly)
    .UseNamingStrategy(new SnakeCaseNamingStrategy())
    .Build();
```

Metoda `Build()` wewnątrz swojej implementacji:

1. Tworzy instancje `ReflectionModelBuilder`.
2. Tworzy instancje `ModelDirector`, przekazując mu budowniczego.
3. Zleca Dyrektorowi konstrukcje modelu dla każdego `Assembly`.
4. Scala wyniki w gotowy `MetadataStore`.

Jadro Systemu: DbContext i Zarządzanie Stanem

Moduł **Core** stanowi centrum operacyjne systemu ORM. To tutaj zbiegają się wszystkie pozostałe warstwy: konfiguracja, metadane, generowanie zapytań oraz dostęp do bazy danych. Głównym zadaniem tej warstwy jest orkiestracja procesów biznesowych (CRUD) oraz zapewnienie spójności danych w pamięci aplikacji.

11 Rola DbContext

Klasa **DbContext** implementuje dwa fundamentalne wzorce architektury systemów korporacyjnych: **Unit of Work** oraz **Identity Map**.

11.1 Wzorzec Unit of Work

DbContext traktuje zestaw operacji na bazie danych jako jedną, niepodzielną jednostkę pracy. Zmiany wprowadzane przez użytkownika (dodawanie, usuwanie, modyfikacja obiektów) nie są wysyłane do bazy danych natychmiast. Zamiast tego, są one buforowane w pamięci przez **ChangeTracker**. Dopiero wywołanie metody **SaveChanges()** powoduje otwarcie transakcji bazodanowej i zatwierdzenie wszystkich zmian naraz. Gwarantuje to atomowość operacji – albo wszystkie zmiany zostaną zapisane, albo żadna.

11.2 Wzorzec Identity Map (First-Level Cache)

System gwarantuje, że w ramach jednego kontekstu (**DbContext**), dana encja o określonym kluczu głównym jest reprezentowana przez dokładnie jedną instancję obiektu.

Implementacja metody **Find<T>** realizuje ten wzorzec w dwóch krokach:

1. **Sprawdzenie pamięci lokalnej:** Metoda najpierw przeszukuje **ChangeTracker**. Jeśli obiekt o zadanym ID został już załadowany lub dodany w bieżącej sesji, jest zwracany natychmiast, bez wykonywania zapytania do bazy danych.
2. **Zapytanie do bazy:** Dopiero w przypadku braku obiektu w pamięci, generator SQL buduje zapytanie **SELECT**, a wynik jest materializowany i natychmiast rejestrowany w **ChangeTracker** (jako *Unchanged*).

Listing 1: Implementacja Identity Map w metodzie **Find**

```
public T? Find<T>(object id) where T : class
{
    // 1. Sprawdzenie Identity Map (ChangeTracker)
    foreach (var entry in ChangeTracker.Entries)
    {
        if (entry.Entity is T entity && entry.State != EntityState.
            Deleted)
```



```

    {
        var map = _configuration.MetadataStore.GetMap<T>();
        var keyVal = map.KeyProperty.PropertyInfo.GetValue(entity);
        if (keyVal != null && keyVal.Equals(id))
            return entity; // Zwróć z pamięci (Cache Hit)
    }
}

// 2. Jeśli nie znaleziono - pobierz z bazy (Cache Miss)
// ... generowanie SELECT ...
}

```

12 Śledzenie Zmian (ChangeTracker)

Mechanizm śledzenia zmian odpowiada za monitorowanie stanu obiektów domenowych. W przeciwieństwie do ciężkich rozwiązań opartych na dynamicznych proxy (jak w starszych wersjach Entity Framework), zastosowano tutaj podejście oparte na jawnej rejestracji encji oraz manualnym zarządzaniu ich stanem (Added, Modified, Deleted). Mechanizm nie wykorzystuje dynamicznych proxy ani snapshotów wartości właściwości..

12.1 Stany encji (EntityState)

Każdy obiekt śledzony przez kontekst znajduje się w jednym z pięciu stanów:

- **Detached:** Obiekt nie jest śledzony przez kontekst.
- **Unchanged:** Obiekt jest zsynchronizowany z bazą danych (np. tuż po pobraniu).
- **Added:** Obiekt jest nowy i oczekuje na wstawienie do bazy (INSERT).
- **Modified:** Obiekt został zmieniony i oczekuje na aktualizację (UPDATE).
- **Deleted:** Obiekt oczekuje na usunięcie z bazy (DELETE).

Klasa `ChangeTracker` przechowuje te informacje w słowniku, mapując referencje do obiektu na obiekt `EntityEntry`, który jest „kopertą” przechowującą metadane stanu.

12.2 Proces Zapisu (SaveChanges)

Metoda `SaveChanges` jest punktem kulminacyjnym cyklu życia kontekstu. Jej algorytm działania wygląda następująco:

1. **Analiza zmian:** System iteruje po wszystkich wpisach w `ChangeTracker`.
2. **Otwarcie transakcji:** Rozpoczyna się transakcja bazodanowa (`IDbTransaction`).

3. **Generowanie komend:** Dla każdego obiektu o stanie innym niż `Unchanged`, odpowiednia metoda `ISqlGenerator` tworzy zapytanie SQL (`INSERT`, `UPDATE` lub `DELETE`).
4. **Obsługa `AutoIncrement`:** W przypadku operacji `INSERT` na bazie `SQLite`, system automatycznie wykonuje dodatkowe zapytanie (`SELECT last_insert_rowid()`), aby pobrać wygenerowane przez bazę ID i uzupełnić nim obiekt w pamięci `C#` przy użyciu refleksji.
5. **Commit:** Zatwierdzenie transakcji.
6. **AcceptAllChanges:** Po udanym zapisie, stan wszystkich obiektów `Added` i `Modified` jest zmieniany na `Unchanged`, a obiekty `Deleted` są usuwane z pamięci trackera.

13 Fasada Dostępu (DbSet)

Klasa `DbSet<T>` pełni rolę fasady oraz adaptera, łączącego świat obiektowy ze światem zapytań. Nie przechowuje ona danych, lecz deleguje operacje do odpowiednich podsystemów:

- **Jako Query Builder:** Implementuje interfejs `IQueryable<T>`. Każde wywołanie metody LINQ (np. `Where`) na obiekcie `DbSet` nie wykonuje zapytania, lecz rozbudowuje wewnętrzne *Expression Tree*. Wykonanie następuje dopiero przy iteracji (np. `foreach` lub `ToList`), kiedy to `QueryProvider` tłumaczy drzewo na SQL.
- **Jako interfejs CRUD:** Metody `Add`, `Update`, `Remove` są skrótami, które delegują zadanie bezpośrednio do `ChangeTracker`'a, ustawiając odpowiedni stan encji (`Added`, `Modified`, `Deleted`).

Dzięki takiemu podziałowi odpowiedzialności, `DbSet` pozostaje lekkim obiektem, który programista może łatwo instancjonować i używać, nie martwiąc się o skomplikowaną logikę znajdującą się pod spodem.

Warstwa Przetwarzania Zapytań (Query Layer)

Jedną z kluczowych funkcjonalności nowoczesnych systemów ORM jest możliwość formułowania zapytań do bazy danych przy użyciu języka obiektowego, zamiast ręcznego klejenia łańcuchów tekstowych SQL. W ekosystemie .NET standardem w tym zakresie jest **LINQ** (Language Integrated Query).

Moduł `src/Query` odpowiada za translację Drzew Wyrażeń (*Expression Trees*) generowanych przez kompilator C# na struktury zrozumiałe dla silnika bazy danych.

14 Architektura Providera LINQ

Punktem styku między aplikacją a mechanizmem zapytań jest klasa `QueryProvider`, implementująca interfejs `System.Linq.IQueryProvider`. Jej działanie opiera się na mechanizmie **Odroczonego Wykonania** (*Deferred Execution*).

Proces budowania zapytania przebiega następująco:

1. **Inicjalizacja:** Użytkownik odwołuje się do `context.Set<User>()`. Zwracany jest obiekt `DbSet`, który zawiera wyrażenie stałe (*ConstantExpression*) wskazujące na źródło danych.
2. **Budowanie (The Chain):** Każde wywołanie metody LINQ (np. `.Where()`, `.OrderBy()`) nie powoduje wykonania kodu. Zamiast tego, `QueryProvider.CreateQuery` tworzy nową instancję `IQueryable`, która opakowuje poprzednie wyrażenie nowym węzłem drzewa (*MethodCallExpression*).
3. **Egzekucja:** Dopiero wywołanie metody wymuszającej iterację (np. `ToList()`, `First()`, lub pętla `foreach`) uruchamia metodę `Execute`. W tym momencie kompletne drzewo wyrażeń jest przekazywane do tłumacza.

15 Translacja Drzew Wyrażeń (SqlExpressionVisitor)

Sercem silnika zapytań jest klasa `SqlExpressionVisitor`, będąca implementacją behawioralnego wzorca projektowego **Wizytator** (*Visitor Pattern*).

Drzewo wyrażeń jest strukturą hierarchiczną, w której każdy węzeł reprezentuje operację (np. dodawanie, porównanie, wywołanie metody) lub operand (zmienna, stała). Klasa `SqlExpressionVisitor` rekurencyjnie „odwiedza” każdy węzeł drzewa, transformując go na fragment zapytania SQL.

15.1 Obsługa metod LINQ

Wizytator rozpoznaje specyficzne metody rozszerzające LINQ i mapuje je na odpowiednie klauzule SQL:

- `Where(predicate)` → Generuje klauzule `WHERE`, rekurencyjnie odwiedzając predykat.
- `OrderBy(selector)` → Generuje klauzule `ORDER BY`.
- `Take(n)` → Tłumaczone na klauzule `LIMIT n` (w dialekcie SQLite).
- `Skip(n)` → Tłumaczone na klauzule `OFFSET n`.

Listing 2: Fragment implementacji wizytatora obsługujący metody LINQ

```
protected override Expression VisitMethodCall(MethodCallExpression node)
{
    if (node.Method.Name == "Where")
    {
        Visit(node.Arguments[0]); // Odwiedź rdo (rekurencja w
            g r )
        _sql.Append(" WHERE ");
        Visit(node.Arguments[1]); // Odwiedź warunek (np. x.Age > 18)
        return node;
    }
    // ... obsługa OrderBy, Take, Skip ...
    return base.VisitMethodCall(node);
}
```

15.2 Translacja Operatorów

Operatory logiczne i arytmetyczne języka `C#` są mapowane na ich odpowiedniki w `SQL`. Odpowiada za to metoda `VisitBinary`.

```
protected override Expression VisitBinary(BinaryExpression node)
{
    Visit(node.Left); // Lewa strona wyrażenia
    _sql.Append(node.NodeType switch
    {
        ExpressionType.Equal => " = ",
        ExpressionType.GreaterThan => " > ",
        ExpressionType.LessThan => " < ",
        ExpressionType.AndAlso => " AND ",
        ExpressionType.OrElse => " OR ",
        _ => throw new NotSupportedException($"Operator {node.NodeType}
            not supported")
    });
    Visit(node.Right); // Prawa strona wyrażenia
    return node;
}
```

16 Bezpieczeństwo i Parametryzacja

Kluczowym aspektem implementacji translatora jest ochrona przed atakami typu *SQL Injection*. Naiwna implementacja polegająca na sklejeniu wartości stałych (np. `x.Name == "Admin"`) bezpośrednio do łańcucha SQL byłaby niebezpieczna.

Zamiast tego, `SqlExpressionVisitor` implementuje automatyczną parametryzację w metodzie `VisitConstant`:

1. Gdy wizytator napotyka węzeł `ConstantExpression` (stała wartość), nie wpisuje jej do tekstu zapytania.
2. Wartość ta jest dodawana do wewnętrznej listy parametrów (`_parameters`).
3. Do tekstu zapytania wstawiany jest placeholder (np. `@p0`, `@p1`).

Dzięki temu zapytanie LINQ: `context.Users.Where(u => u.Name == userInput)` zostanie przetłumaczone na bezpieczny SQL: `SELECT * FROM Users WHERE Name = @p0`, gdzie wartość `userInput` zostanie przesłana do bazy danych osobnym kanałem przez sterownik ADO.NET.

17 Model Pośredni (QueryModel)

Dla bardziej złożonych zapytań, wykraczających poza proste filtrowanie (np. złączenia JOIN, grupowanie), architektura przewiduje wykorzystanie obiektu pośredniego `QueryModel` (implementacja wzorca *Parameter Object* oraz *Composite*).

Zamiast generować tekst SQL bezpośrednio w wizytatorze, zaawansowany translator buduje strukturę `QueryModel`, która przechowuje:

- Listę kolumn do pobrania (`SelectColumns`).
- Listę złączeń (`Joins`).
- Warunki logiczne.
- Funkcje agregujące.

Takie podejście (opisane w sekcji 6 dokumentacji źródłowej) pozwala na lepszą separację logiki translacji od generowania konkretnego dialektu SQL, co ułatwia ewentualne wsparcie dla innych baz danych w przyszłości.

Warstwa Generowania SQL (SQL Generation Layer)

Warstwa generowania SQL pełni rolę tłumacza pomiędzy logiczną reprezentacją operacji w systemie ORM (metadane, obiekty domenowe, modele zapytań) a tekstowym językiem zapytań zrozumiałym dla silnika bazy danych.

Zaprojektowana architektura realizuje zasadę odwrócenia zależności (Dependency Inversion Principle) – wyższe warstwy systemu nie zależą bezpośrednio od specyfiki bazy SQLite, lecz od abstrakcyjnego interfejsu generatora.

18 Abstrakcja Generatora (ISqlGenerator)

Interfejs `ISqlGenerator` definiuje kontrakt dla wszystkich operacji manipulacji danymi (DML) oraz definicji danych (DDL). Zastosowano tu wzorzec **Strategia** (*Strategy Pattern*), gdzie kontekstem jest `DbContext`, a strategia konkretna implementacja dla danej bazy danych.

Generator uwzględnia również informacje o hierarchii dziedziczenia encji zawarte w `EntityMap`, co pozwala na prawidłowe mapowanie klas bazowych i pochodnych zgodnie z wybraną strategią dziedziczenia (*Inheritance Strategy Pattern*).

Interfejs wymusza implementację metod dla pełnego spektrum operacji CRUD:

- `GenerateSelect` – generowanie pobierania po kluczu głównym.
- `GenerateInsert` – generowanie wstawiania rekordów.
- `GenerateUpdate` – generowanie aktualizacji istniejących rekordów.
- `GenerateDelete` – generowanie usuwania rekordów.
- `GenerateComplexSelect` – generowanie złożonych zapytań z `QueryModel` (obsługa JOIN, GroupBy).

Dodatkowo, interfejs abstrahuje różnice w dialektach SQL, takie jak sposób cytowania identyfikatorów (np. "Tabela" vs [Tabela]) czy prefiksy parametrów.

19 Implementacja dla SQLite (SqliteSqlGenerator)

Klasa `SqliteSqlGenerator` jest konkretną implementacją strategii dostosowaną do specyfiki bazy SQLite. Zawiera ona logikę niezbędną do poprawnej komunikacji z tym silnikiem.

Przy generowaniu zapytań SQL, implementacja bierze pod uwagę metadane z `EntityMap`, w tym informacje o dziedziczeniu klas. Pozwala to na automatyczne dołączanie kolumn i tabel klas bazowych podczas operacji na encjach pochodnych.

19.1 Obsługa Autoinkrementacji (Identity)

Jednym z krytycznych aspektów implementacji jest obsługa kluczy głównych generowanych przez bazy danych. SQLite nie wspiera składni `RETURNING` w starszych wersjach, dlatego generator stosuje technikę doklejania drugiego zapytania.

W metodzie `GenerateInsert`, jeśli encja posiada klucz typu `AutoIncrement`, generator tworzy zapytanie złożone:

```
INSERT INTO "Users" ("Name", "Email") VALUES (@Name, @Email);
SELECT last_insert_rowid();
```

Pozwala to na natychmiastowe odczytanie wygenerowanego ID i zaktualizowanie obiektu w pamięci aplikacji w ramach jednej operacji `ExecuteScalar`.

19.2 Cytowanie i Parametry

Generator dba o poprawność składniową i bezpieczeństwo:

- **Cytowanie:** Wszystkie nazwy tabel i kolumn są otaczane cudzysłowami (np. `"User"`), co pozwala na używanie słów kluczowych SQL jako nazw właściwości w C#.
- **Parametryzacja:** Zamiast wstawiać wartości bezpośrednio do ciągu SQL, generator tworzy słownik parametrów (np. `@Age`, `@Name`). Wartości `null` są automatycznie konwertowane na `DBNull.Value`, co jest wymagane przez ADO.NET.

20 Budowanie Zapytań (SqlQueryBuilder)

Aby uniknąć podatnego na błędy ręcznego sklejanego łańcuchów znaków (String Concatenation), zaimplementowano pomocniczą klasę `SqlQueryBuilder`, działającą zgodnie z wzorcem **Builder** oraz udostępniającą interfejs płynny (*Fluent Interface*).

Klasa ta kapsułkuje `StringBuilder` i udostępnia metody odpowiadające klauzulom SQL, dbając o poprawne formatowanie (spacje, przecinki).

Listing 3: Przykład użycia `SqlQueryBuilder` wewnątrz generatora

```
var builder = new SqlQueryBuilder();
builder.Select(columnsFiltered)
    .From(map.TableName)
    .Where($"{map.KeyProperty.ColumnName} = @id");
```

`SqlQueryBuilder` wspiera również zaawansowane konstrukcje, takie jak:

- **JOIN:** Obsługa złączeń `INNER`, `LEFT`, `RIGHT`, `FULL OUTER` wraz z aliasami tabel. Mechanizm ten jest wykorzystywany również do mapowania hierarchii dziedziczenia, gdzie tabele klas bazowych są automatycznie dołączane do zapytań.

- **Paginacja:** Generowanie klauzul `LIMIT` i `OFFSET` specyficznych dla SQLite.
- **Grupowanie:** Obsługa `GROUP BY` oraz `HAVING`.

Dzięki takiemu podejściu, kod generatora jest czytelny, łatwy w utrzymaniu i mniej podatny na błędy składniowe SQL.

Warstwa Materializacji Obiektów (Materialization Layer)

Proces materializacji (*Object Hydration*) jest jednym z najbardziej krytycznych punktów w każdym systemie ORM pod względem wydajności. Polega on na przekształceniu surowych danych zwróconych przez bazę danych (interfejs `IDataRecord`) na instancje obiektów domenowych.

Nasz system wykorzystuje kompilowane w locie Drzewa Wyrażeń, dzięki czemu nie generuje znaczącego narzutu na procesor. (*Expression Trees*).

21 Rola ObjectMaterializer

Klasa `ObjectMaterializer` pełni rolę bezstanowej fabryki obiektów. Jej działanie opiera się na zasadzie: „*Przeanalizuj typ raz, wykonuj wielokrotnie*”.

W konstruktorze materializera następuje kosztowny proces przygotowawczy:

1. Pobierana jest definicja `EntityMap` oraz strategia dziedziczenia.
2. Generowany jest kod odpowiedzialny za tworzenie instancji (konstruktor).
3. Generowany jest kod odpowiedzialny za przypisywanie wartości do właściwości (settery).
4. Wygenerowany kod jest kompilowany do szybkich delegatów i cache'owany.

Dzięki temu, podczas przetwarzania tysięcy rekordów w pętli `while (reader.Read())`, system nie używa refleksji, lecz wywołuje gotowe, skompilowane funkcje.

22 Obsługa Polimorfizmu (Inheritance Support)

`ObjectMaterializer` obsługuje polimorfizm w czasie rzeczywistym. Przed utworzeniem instancji obiektu, materializer analizuje `InheritanceStrategy` mapy:

- Jeśli wykryta zostanie strategia **TPH (Table Per Hierarchy)**, materializer odczytuje wartość kolumny dyskryminatora z bieżącego wiersza.
- Na podstawie tej wartości dobierana jest odpowiednia mapa dla klasy pochodnej (np. `Dog` zamiast `Animal`).
- Następuje przekazanie sterowania do zcache'owanego materializera właściwego dla wykrytego typu konkretnego.

Dzięki temu zapytanie o listę zwierząt (`context.Animals.ToList()`) poprawnie zwraca kolekcję zawierającą instancje psów i kotów.

23 Optymalizacja z użyciem Expression Trees

Zamiast tradycyjnej refleksji, zastosowano metaprogramowanie. Poniżej przedstawiono mechanizmy optymalizacyjne zaimplementowane w klasie `ObjectMaterializer`.

23.1 Szybka Fabryka Obiektów

Tworzenie nowej instancji encji odbywa się poprzez delegat `Func<object>`. Kod generujący ten delegat wygląda następująco:

```
private static Func<object> CreateFactory(Type type)
{
    var ctor = Expression.New(type); // Odpowiednik: new User()
    var convert = Expression.Convert(ctor, typeof(object));
    // Kompilacja do kodu maszynowego:
    return Expression.Lambda<Func<object>>(convert).Compile();
}
```

Wydajność takiego delegata jest zbliżona do bezpośredniego wywołania operatora `new` w kodzie C#.

23.2 Kompilowane Settery

Dla każdej właściwości mapowanej (*Scalar Property*), materializer buduje dedykowaną akcję `Action<object, object?>`. Kod ten wykonuje rzutowanie typów i przypisanie wartości, omijając narzut metody `SetValue`.

24 Bezstanowość i Indeksy Kolumn (Ordinals)

Kolejna istotna decyzja architektoniczna jest uniezależnienie materializera od kolejności kolumn w zapytaniu SQL. Materializer nie zakłada, że kolumna "Id" jest zawsze na pozycji 0.

Zamiast tego, metoda `Materialize` przyjmuje tablice indeksów (`int[] ordinals`), która jest obliczana przez `QueryProvider` na podstawie aktualnego rezultatu zapytania `IDataReader`.

```
public object Materialize(IDataRecord record, int[] ordinals)
{
    // 1. Obsługa Polimorfizmu (TPH)
    if (_rootMap.InheritanceStrategy is TablePerHierarchyStrategy tph)
    {
        int discOrdinal = GetDiscriminatorOrdinal(record, tph.
            DiscriminatorColumn);
        if (discOrdinal >= 0)
    }
}
```

```

    {
        string discValue = record.GetString(discOrdinal);
        // Je li to typ pochodny, przeka do odpowiedniego
        materializera
        if (discValue != tph.DiscriminatorValue)
        {
            return MaterializeDerived(record, discValue);
        }
    }
}

// 2. Standardowa materializacja
var instance = _factory(); // Szybkie utworzenie obiektu
int index = 0;

foreach (var prop in _rootMap.ScalarProperties)
{
    if (index >= ordinals.Length) break;
    int ordinal = ordinals[index++];

    if (ordinal < 0) continue;

    if (!_setters.TryGetValue(prop, out var setter)) continue;

    object? value = record.IsDBNull(ordinal) ? null : record.
        GetValue(ordinal);

    // Konwersja typ w (np. Enum, Guid) i ustawienie warto ci
    if (value != null) value = ConvertValue(value, prop);
    setter(instance, value);
}
return instance;
}

```

25 Obsługa Typów i Bezpieczeństwo

Materializer dba o spójność typów pomiędzy baza danych (SQL) a środowiskiem .NET:

- **DBNull vs null:** Wartość `DBNull.Value` z bazy jest automatycznie konwertowana na `null`.
- **Typy Enum:** Wartości liczbowe z bazy są rzutowane na odpowiednie typy wyliczeniowe przy użyciu `Enum.ToObject`.
- **Walidacja:** System rzuca wyjątek `InvalidOperationException`, jeśli baza zwraca `NULL` dla kolumny, która w modelu C# jest typem wartościowym nienullowalnym

(np. `int`), co zapobiega cichym błedom i niespójności danych.

Analiza Zastosowanych Wzorców Projektowych

Projekt systemu ORM stanowił doskonałe pole do praktycznego zastosowania klasycznych wzorców projektowych (GoF) oraz wzorców architektury systemów korporacyjnych (PoEAA). Każdy wzorzec został użyty świadomie, w celu rozwiązania konkretnego problemu architektonicznego.

Poniżej przedstawiono analize najważniejszych wzorców wraz z uzasadnieniem ich wyboru.

26 Wzorzec Builder (Budowniczy)

W module mapowania obiektowo-relacyjnego zastosowano wzorzec projektowy *Builder* w celu odseparowania procesu konstrukcji złożonego grafu metadanych (*EntityMap*) od jego reprezentacji. Implementacja wzorca pozwala na kontrolowane tworzenie modelu z uwzględnieniem zależności dziedziczenia oraz umożliwia łatwa wymianę strategii budowania bez modyfikacji kodu klienta.

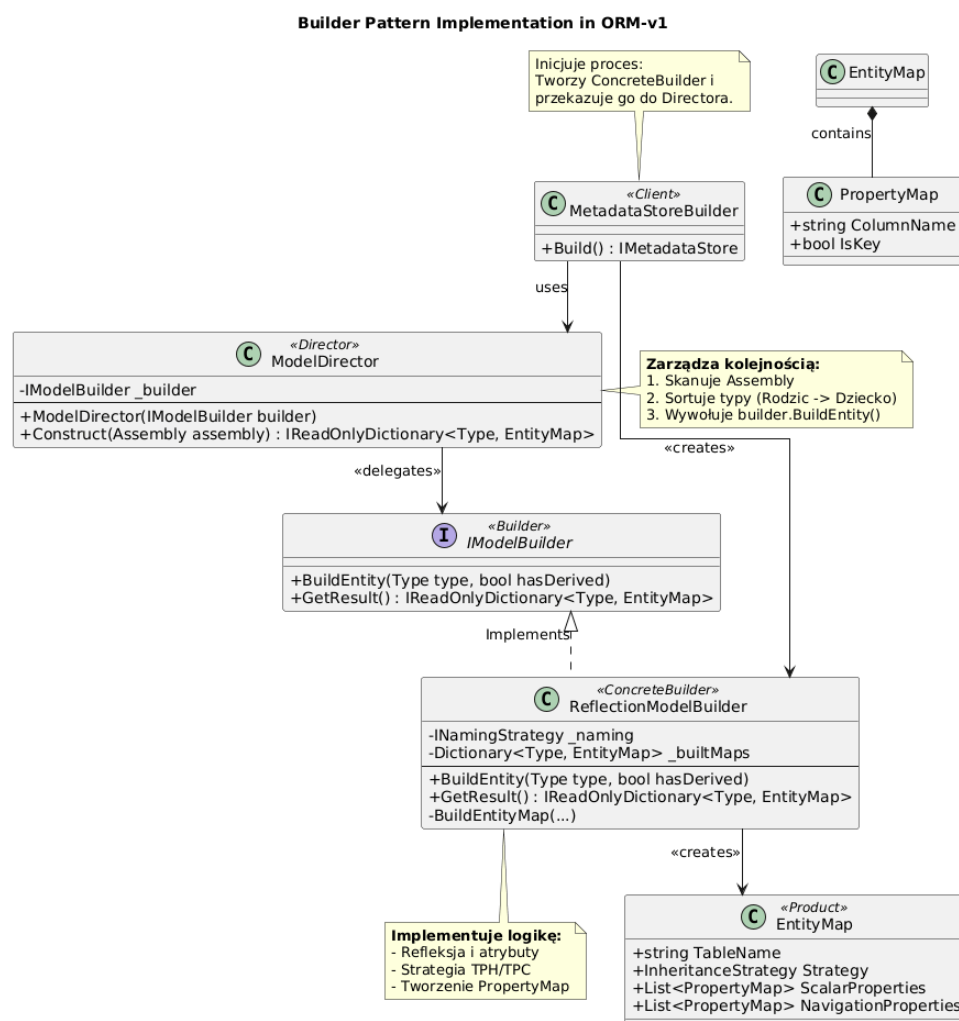


Figure 3: Diagram klas implementacji wzorca Builder w ORM-v1

Mapowanie ról wzorca w projekcie

Poniżej przedstawiono realizacje poszczególnych uczestników wzorca w kodzie systemu:

Client: MetadataStoreBuilder Pełni rolę inicjatora procesu. Tworzy instancje konkretnego budowniczego (`ReflectionModelBuilder`) oraz dyrektora (`ModelDirector`), a następnie uruchamia procedurę konstrukcji. Odbiera gotowy wynik i zamyka go w obiekcie `MetadataStore`.

Director: ModelDirector Odpowiada za sterowanie procesem budowy (algorytm konstrukcji). Nie zajmuje się analizą atrybutów, lecz zarządza kolejnością operacji:

- Skanuje wskazane `Assembly` w poszukiwaniu klas encji.
- Sortuje typy topologicznie (od rodzica do dziecka), aby zapewnić poprawność mapowania dziedziczenia.
- Iteracyjnie zleca budowniczemu utworzenie mapy dla każdego typu.

Builder (Interface): IModelBuilder Abstrakcyjny interfejs definiujący kontrakt budowania:

- `BuildEntity(Type type, bool hasDerivedTypes)` – metoda budująca pojedynczy element.
- `GetResult()` – metoda zwracająca rezultat pracy.

Concrete Builder: ReflectionModelBuilder Implementuje szczegóły techniczne tworzenia mapy przy użyciu mechanizmu Refleksji:

- Analizuje atrybuty metadanych (`[Table]`, `[Key]`).
- Podejmuje decyzje o strategii mapowania dziedziczenia (TPH vs TPC).
- Mapuje właściwości klasy na kolumny bazy danych (`PropertyMap`).

Product: EntityMap Złożony obiekt wynikowy reprezentujący pełną definicję mapowania tabeli. Jest to obiekt niemutowalny (immutable) po utworzeniu, zawierający komplet informacji niezbędnych dla generatora SQL i materializera obiektów.

Zalety

- **Separacja odpowiedzialności (SRP):** Sterowanie procesem budowy modelu (kolejność, dziedziczenie) zostało oddzielone od logiki technicznej mapowania (refleksja, atrybuty), co zwiększa czytelność i utrzymywalność kodu.
- **Spójność tworzonych obiektów:** Wzorec Builder umożliwia konstruowanie złożonego obiektu `EntityMap` etapami i zwrócenie go dopiero w pełni poprawnym stanie, eliminując ryzyko użycia niekompletnych metadanych.

- **Rozszerzalność (OCP):** Możliwe jest dodanie nowych sposobów budowy modelu (np. z plików konfiguracyjnych) poprzez implementację `IModelBuilder`, bez modyfikacji istniejącego kodu systemu.
- **Kontrola nad dziedziczeniem:** Centralne sterowanie procesem budowy gwarantuje poprawną kolejność tworzenia map klas bazowych i pochodnych, co jest kluczowe dla strategii TPH i TPC.

Wady

- **Większa złożoność struktury:** Zastosowanie wzorca wymaga wprowadzenia dodatkowych klas i interfejsów, co w małych projektach może być postrzegane jako nadmierna komplikacja.
- **Powiązanie z produktem:** Konkretny budowniczy jest ściśle związany ze strukturą `EntityMap`, dlatego zmiany w modelu danych wymagają aktualizacji logiki budowy.
- **Błędy wykrywane w czasie wykonania:** Wykorzystanie refleksji powoduje, że część błędów konfiguracyjnych ujawnia się dopiero podczas budowy modelu, a nie na etapie kompilacji.

27 Wzorzec Unit of Work (Jednostka Pracy)

Wzorzec *Unit of Work* został zastosowany w celu zarządzania transakcjami bazodanowymi oraz zapewnienia spójności operacji wykonywanych na wielu encjach. W projekcie ORM-v1 wzorzec ten pełni kluczową rolę w klasie `DbContext`, która śledzi zmiany w obiektach i koordynuje ich zapis do bazy danych w ramach pojedynczej transakcji.

Dzięki zastosowaniu tego wzorca, aplikacja może grupować wiele operacji (wstawianie, aktualizacja, usuwanie) i zatwierdzać je jednocześnie poprzez wywołanie metody `SaveChanges()`, co gwarantuje atomowość i spójność danych.

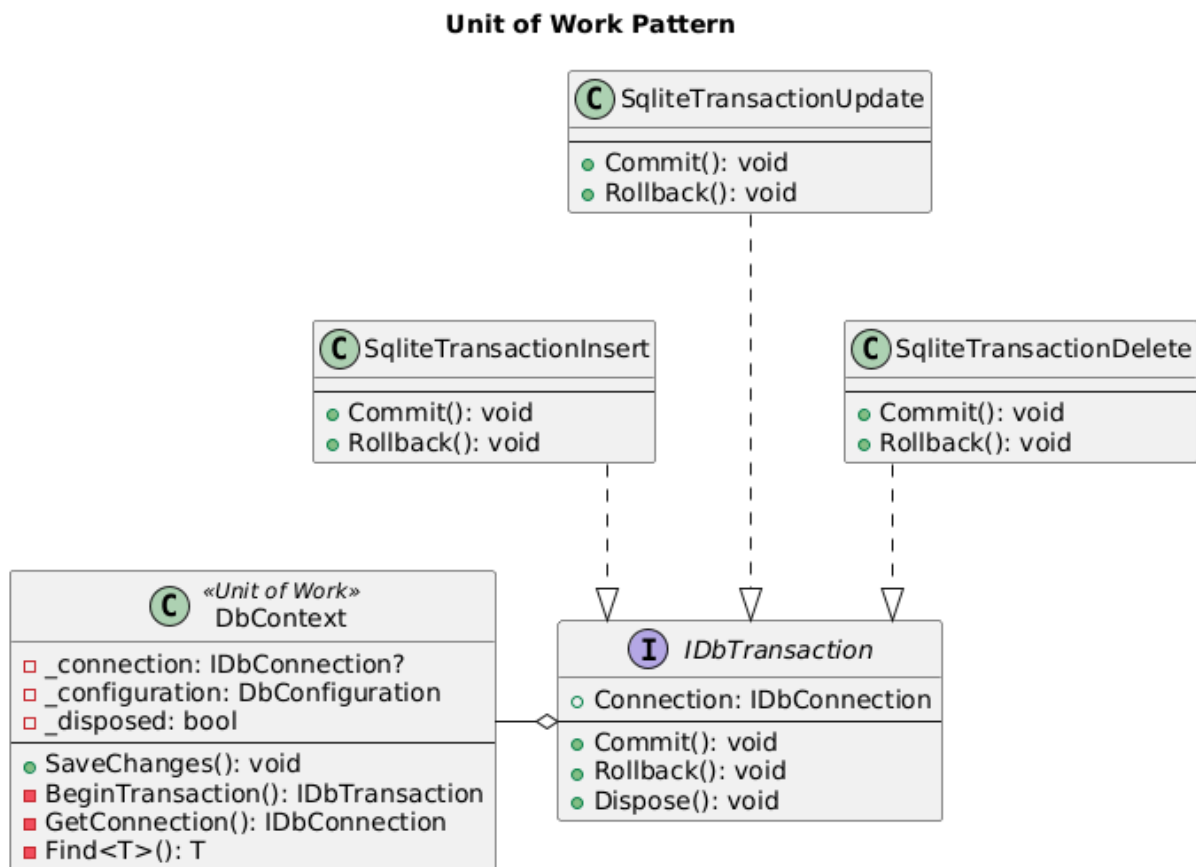


Figure 4: Diagram klas implementacji wzorca Unit of Work

Mapowanie ról wzorca w projekcie

Poniżej przedstawiono realizacje komponentów wzorca w kontekście zarządzania transakcjami:

Unit of Work: DbContext Klasa koordynująca wszystkie operacje na encjach w ramach sesji pracy. Przechowuje referencje do połączenia bazodanowego (`_connection: IDbConnection`) oraz konfiguracji (`_configuration: DbConfiguration`). Zapewnia metody do inicjowania transakcji (`BeginTransaction()`), pobierania połączenia (`GetConnection()`) oraz wyszukiwania encji (`Find<T>()`). Metoda `SaveChanges()` stanowi centralny punkt zatwierdzania wszystkich nagromadzonych zmian.

Transaction Interface: IDbTransaction Interfejs definiujący wspólny kontrakt dla wszystkich typów transakcji bazodanowych.

- `Connection: IDbConnection` – właściwość udostępniająca połączenie bazodanowe.
- `Commit(): void` – metoda zatwierdzająca transakcje.

- `Rollback()`: `void` – metoda wycofująca zmiany w przypadku błędu.
- `Dispose()`: `void` – metoda zwalniająca zasoby transakcji.

Concrete Transactions: Implementacje specyficznych operacji transakcyjnych:

- `SqliteTransactionInsert` – Enkapsuluje logikę transakcji wstawiania nowych rekordów do bazy danych. Implementuje mechanizm zatwierdzania i wycofywania operacji INSERT.
- `SqliteTransactionUpdate` – Realizuje transakcje aktualizacji istniejących rekordów. Zapewnia możliwość modyfikacji danych z zachowaniem atomowości operacji.
- `SqliteTransactionDelete` – Obsługuje transakcje usuwania rekordów z bazy. Umożliwia bezpieczne wycofanie operacji DELETE w przypadku wystąpienia błędu.

Zalety

- **Atomowość operacji:** Wzorzec gwarantuje, że wszystkie operacje w ramach jednostki pracy zostaną zatwierdzone jednocześnie lub w całości wycofane w przypadku błędu, co zapobiega niespójności danych w bazie.
- **Centralizacja zarządzania transakcjami:** Klasa `DbContext` stanowi pojedynczy punkt kontroli dla wszystkich operacji bazodanowych, co upraszcza kod klienta i eliminuje konieczność ręcznego zarządzania transakcjami w każdym miejscu użycia.
- **Optymalizacja wydajności:** Grupowanie wielu operacji w jedną transakcję redukuje liczbę połączeń z bazą danych i zmniejsza narzut komunikacyjny, co poprawia wydajność aplikacji, szczególnie przy operacjach wsadowych.
- **Separacja odpowiedzialności:** Konkretne klasy transakcji (`SqliteTransactionInsert`, `SqliteTransactionUpdate`, `SqliteTransactionDelete`) enkapsulują specyficzną logikę dla różnych typów operacji, co realizuje zasadę pojedynczej odpowiedzialności (SRP).
- **Łatwość testowania:** Interfejs `IDbTransaction` umożliwia podstawienie mock'ów w testach jednostkowych, co ułatwia weryfikację logiki biznesowej bez konieczności korzystania z rzeczywistej bazy danych.

Wady

- **Zwiększona złożoność:** Wprowadzenie wzorca wymaga stworzenia dodatkowej warstwy abstrakcji (interfejsy, klasy transakcji), co zwiększa złożoność architektury systemu i może być nadmiarowe w prostych aplikacjach z niewielkimi operacjami bazodanowymi.

- **Narzut pamięciowy:** Śledzenie wszystkich zmian w obiektach wymaga przechowywania dodatkowych informacji o stanie encji w pamięci, co może prowadzić do zwiększonego zużycia zasobów przy pracy z dużą liczbą obiektów.
- **Ryzyko długotrwałych transakcji:** Jeśli jednostka pracy nie zostanie odpowiednio zarządzana (np. zbyt długo otwarta transakcja), może to prowadzić do blokad w bazie danych i pogorszenia współbieżności systemu.
- **Ukryta złożoność SaveChanges:** Metoda `SaveChanges()` może wykonywać wiele operacji jednocześnie, co utrudnia debugowanie i śledzenie przepływu danych, szczególnie gdy wystapia błędy w środku procesu zatwierdzania.

28 Wzorzec Identity Map (Mapa Tożsamości)

Wzorzec *Identity Map* został zastosowany w celu zapewnienia, że każdy obiekt z bazy danych jest ładowany tylko raz w ramach sesji i przechowywany w pamięci podręcznej. W projekcie ORM-v1 wzorzec ten pełni kluczową rolę w klasie `ChangeTracker`, która śledzi wszystkie encje załadowane z bazy danych oraz ich stany, eliminując duplikacje obiektów i zapewniając spójność tożsamości.

Dzięki zastosowaniu tego wzorca, wielokrotne zapytania o ten sam obiekt (na podstawie klucza głównego) zwracają referencje do tej samej instancji w pamięci, co gwarantuje integralność referencyjną i unika problemów związanych z modyfikacją wielu kopii tego samego rekordu.

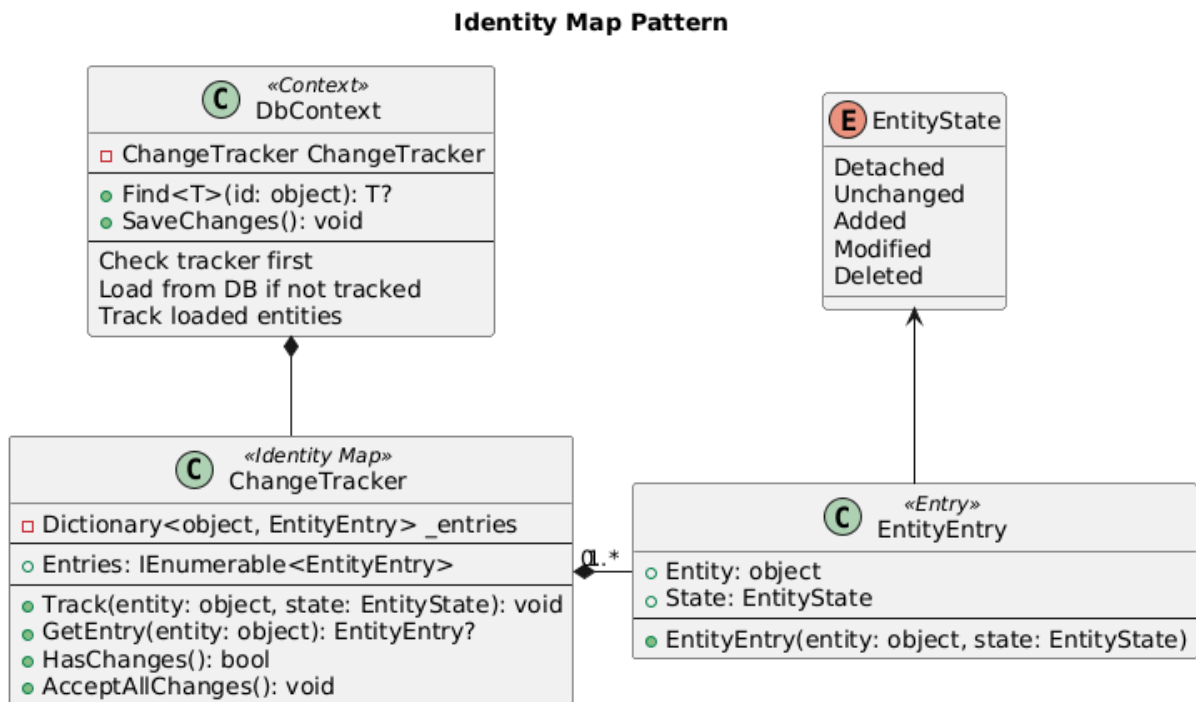


Figure 5: Diagram klas implementacji wzorca Identity Map

Mapowanie ról wzorca w projekcie

Poniżej przedstawiono realizacje komponentów wzorca w kontekście śledzenia tożsamości encji:

Context: DbContext Klasa stanowiąca punkt wejścia do operacji na encjach. Przechowuje referencje do **ChangeTracker** i wykorzystuje go w metodzie **Find<T>(id: object)**, która najpierw sprawdza, czy encja jest już śledzona w pamięci, a dopiero w przypadku braku pobiera ją z bazy danych. Po załadowaniu z bazy, encja jest automatycznie rejestrowana w trackerze. Metoda **SaveChanges()** koordynuje zapis wszystkich śledzonych zmian.

Identity Map: ChangeTracker Klasa implementująca wzorzec mapy tożsamości. Przechowuje słownik **_entries: Dictionary<object, EntityEntry>**, który mapuje obiekty encji na ich wpisy śledzące. Zapewnia metody zarządzania śledzonymi encjami:

- **Entries: IEnumerable<EntityEntry>** – kolekcja wszystkich śledzonych wpisów encji.
- **TrackIdentity(object, state: EntityState): void** – rejestruje encje w mapie z określonym stanem.

- `GetEntry(entity: object): EntityEntry?` – pobiera wpis encji z mapy, jeśli istnieje.
- `HasChanges(): bool` – sprawdza, czy są jakieś niezapisane zmiany.
- `AcceptAllChanges(): void` – akceptuje wszystkie zmiany i aktualizuje stany encji.

Entity Entry: EntityEntry Klasa enkapsulująca informacje o pojedynczej śledzonej encji. Zawiera referencje do obiektu encji (`Entity: object`) oraz jej aktualny stan (`State: EntityState`). Konstruktor przyjmuje encję i stan początkowy, tworząc wpis śledzacy dla danego obiektu.

Entity State: EntityState (Enum) Typ wyliczeniowy definiujący możliwe stany encji w kontekście śledzenia zmian:

- `Detached` – encja nie jest śledzona przez kontekst.
- `Unchanged` – encja jest śledzona, ale nie została zmodyfikowana.
- `Added` – nowa encja oczekująca na wstawienie do bazy.
- `Modified` – istniejąca encja z niezapisanymi zmianami.
- `Deleted` – encja oznaczona do usunięcia z bazy.

Zalety

- **Gwarancja tożsamości obiektów:** Wzorzec zapewnia, że wielokrotne odwołania do tego samego rekordu w bazie danych zwracają tę samą instancję obiektu w pamięci, co eliminuje problem niespójności między wieloma kopiami tego samego obiektu.
- **Redukcja zapytań do bazy danych:** Metoda `Find<T>()` najpierw sprawdza tracker, zanim wykona zapytanie do bazy, co znacząco zmniejsza liczbę operacji I/O i poprawia wydajność aplikacji przy wielokrotnym dostępie do tych samych danych.
- **Automatyczne śledzenie zmian:** Wszystkie załadowane encje są automatycznie rejestrowane w `ChangeTracker`, co umożliwia efektywne wykrywanie modyfikacji i generowanie odpowiednich zapytań `UPDATE/DELETE` podczas wywołania `SaveChanges()`.
- **Uproszczenie logiki klienta:** Deweloperzy nie muszą ręcznie zarządzać cache'owaniem obiektów ani martwić się o duplikacje instancji – wzorzec obsługuje to transparentnie w warstwie ORM.

- **Integralność referencyjna w pamięci:** Gdy dwa różne obiekty w aplikacji posiadają relacje do tej samej encji, oba wskazują na te same instancje w pamięci, co zapewnia spójność relacji i ułatwia nawigację po grafie obiektów.

Wady

- **Zwiększone zużycie pamięci:** Wszystkie załadowane encje pozostają w pamięci przez cały czas życia kontekstu, co przy długotrwałych sesjach lub ładowaniu dużych zbiorów danych może prowadzić do znacznego wzrostu zużycia RAM.
- **Złożoność zarządzania cyklem życia:** Konieczność świadomego zarządzania czasem życia kontekstu (wzorzec Unit of Work per Request) oraz odpowiedniego czyszczenia trackera po zakończeniu operacji może prowadzić do błędów typu memory leak, jeśli kontekst nie zostanie prawidłowo zdisposowany.

29 Obsługa Dziedziczenia (Inheritance Mapping)

System ORM implementuje obsługę polimorfizmu w modelu relacyjnym, wykorzystując wzorzec *Strategy*. Decyzja o sposobie mapowania hierarchii klas na tabele bazy danych jest podejmowana na etapie budowania modelu i zamykana w obiekcie implementującym interfejs *IInheritanceStrategy*.

Dzięki temu warstwy generowania SQL oraz materializacji obiektów otrzymują jednoznaczny przepis na obsługę danej encji, niezależnie od skomplikowania hierarchii.

29.1 Obsługiwane Strategie

System wspiera trzy podejścia do mapowania dziedziczenia, reprezentowane przez osobne klasy strategii:

TPH (Table Per Hierarchy) – Domyślna. Cała hierarchia dziedziczenia mapowana jest do jednej, wspólnej tabeli.

- **Model:** Wymaga kolumny dyskryminatora (domyślnie "Discriminator"), która przechowuje nazwę typu (np. "Dog").
- **Zastosowanie:** Najwyższa wydajność odczytu (brak złączeń), ale tabela zawiera kolumny wszystkich klas pochodnych (często puste/nullable).

TPC (Table Per Concrete Class). Każda klasa konkretna (nie abstrakcyjna) posiada własną, niezależną tabelę.

- **Model:** Tabele są niezależne i zawierają komplet kolumn (odziedziczone + własne). Brak wspólnego klucza czy relacji między tabelami w sensie SQL.

- **Zastosowanie:** Dobra izolacja danych, ale zapytania polimorficzne (np. "daj wszystkie Zwierzeta") wymagają kosztownej operacji UNION.

TPT (Table Per Type) – Znormalizowana. Każda klasa w hierarchii posiada własną tabelę, ale zawierającą tylko pola zdefiniowane w tej klasie.

- **Model:** Tabela klasy pochodnej posiada klucz główny będący jednocześnie kluczem obcym do tabeli bazowej (relacja 1:1).
- **Zastosowanie:** Najczystszy model relacyjny (brak redundancji, brak nulli), ale wymaga złączeń (JOIN) przy pobieraniu danych.

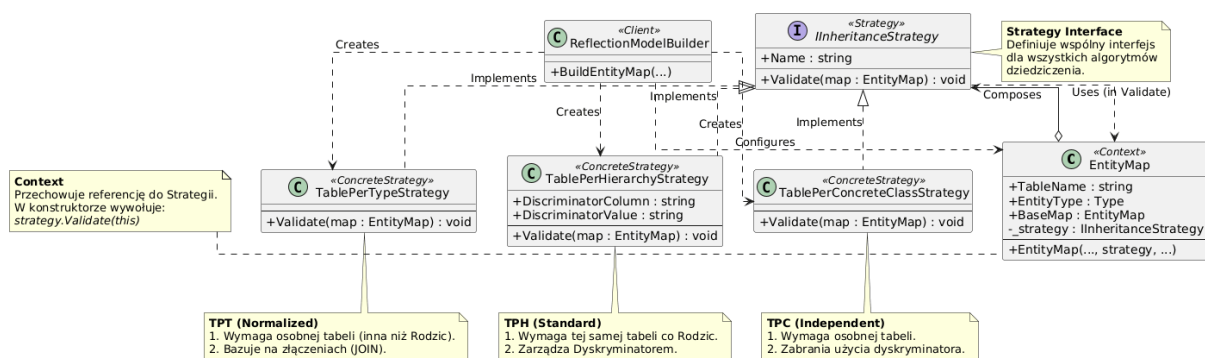


Figure 6: Implementacja strategii dziedziczenia w module Mapping

29.2 Algorytm Doboru Strategii (Heurystyka Buildera)

Moduł budujący model (`ReflectionModelBuilder`) automatycznie dobiera odpowiednią strategię na podstawie analizy atrybutów oraz struktury klas. Proces decyzyjny przebiega następująco:

1. **Analiza Korzenia (Root):** Jeśli klasa bazowa posiada atrybut `[InheritanceStrategy(strategy.TablePerType)]`, cała hierarchia wymuszana jest w tryb **TPT**. Wymaga to, aby każda klasa pochodna posiadała własną tabelę.
2. **Wykrycie TPC (Table Per Concrete Class):** Jeśli klasa pochodna posiada atrybut `[Table("Nazwa")]`, a na korzeniu nie wymuszono **TPT**, system zakłada strategię **TPC**. Interpretuje to jako intencje użytkownika: "Chce mapować tę konkretną klasę do osobnej tabeli".
3. **Domyślne TPH:** W przypadku braku powyższych atrybutów, system przyjmuje strategię **TPH**. Klasa pochodna jest mapowana do tabeli rodzica, a w strategii rejestrowana jest wartość dyskryminatora (nazwa klasy).

4. **Klasy Samodzielne:** Klasy nieposiadające dziedziczenia są traktowane jako tryb **TPC** (pojedyncza tabela konkretna), co jest optymalizacją eliminującą potrzebę kolumny dyskryminatora dla prostych encji.

29.3 Walidacja Spójności Modelu

Każda strategia implementuje metodę `Validate(EntityMap map)`, która jest uruchamiana na końcu procesu budowania. Gwarantuje ona, że wygenerowany model jest logicznie poprawny przed przekazaniem go do pozostałych modułów systemu:

- **Dla TPH:** Weryfikuje, czy tabela klasy pochodnej jest identyczna z tabelą rodzica.
- **Dla TPT:** Weryfikuje, czy tabela klasy pochodnej jest **różna** od tabeli rodzica (wymóg złączenia).
- **Dla TPC/TPT:** Upewnia się, że nie zdefiniowano zbędnych parametrów dyskryminatora, które są specyficzne tylko dla TPH.

W celu lepszego zrozumienia dynamiki systemu, poniżej przedstawiono diagramy sekwencji obrazujące kluczowe scenariusze użycia ORM.

30 Diagram Sekwencji: Inicjalizacja Modelu

Diagram przedstawia proces budowania `MetadataStore` przy starcie aplikacji.

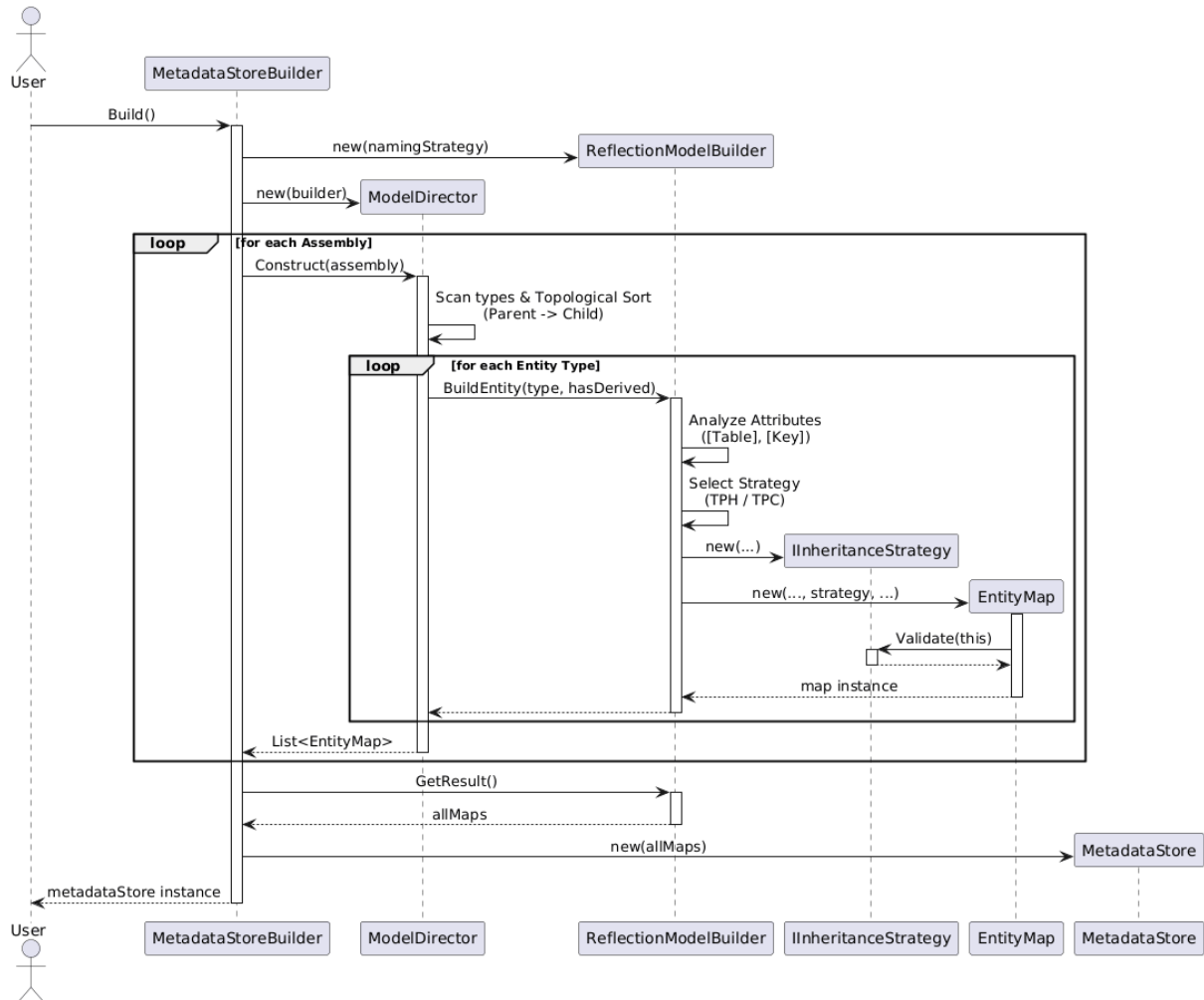


Figure 7: Diagram Sekwencji: Inicjalizacja Modelu

31 Diagram Sekwencji: Wykonanie Zapytania LINQ

Scenariusz pobrania danych: `context.Set<User>().Where(...).ToList()`.

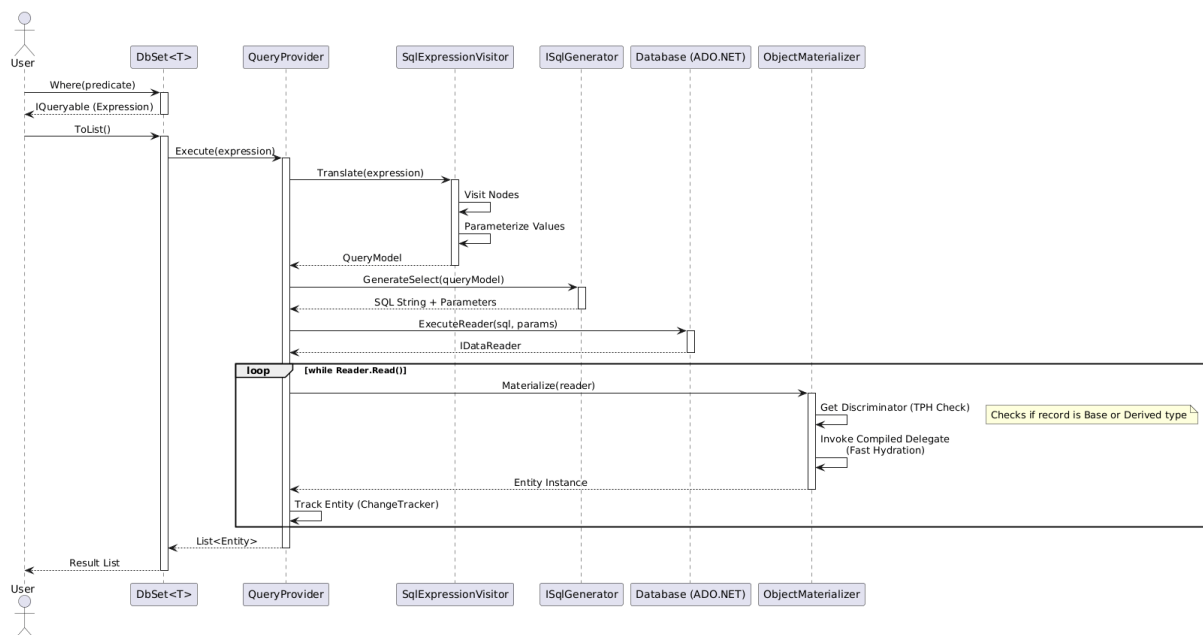


Figure 8: Diagram Sekwencji: Wykonanie Zapytania LINQ

32 Diagram Aktywności (Activity Diagram)

Poniższy diagram obrazuje przepływ sterowania podczas wykonywania zapytania do bazy danych. Diagram wykorzystuje notację z torami (swimlanes), aby wskazać odpowiedzialność poszczególnych warstw systemu za realizację kolejnych kroków procesu – od translacji LINQ, przez wykonanie SQL, aż po materializację wyników i rejestrację w **ChangeTracker**.

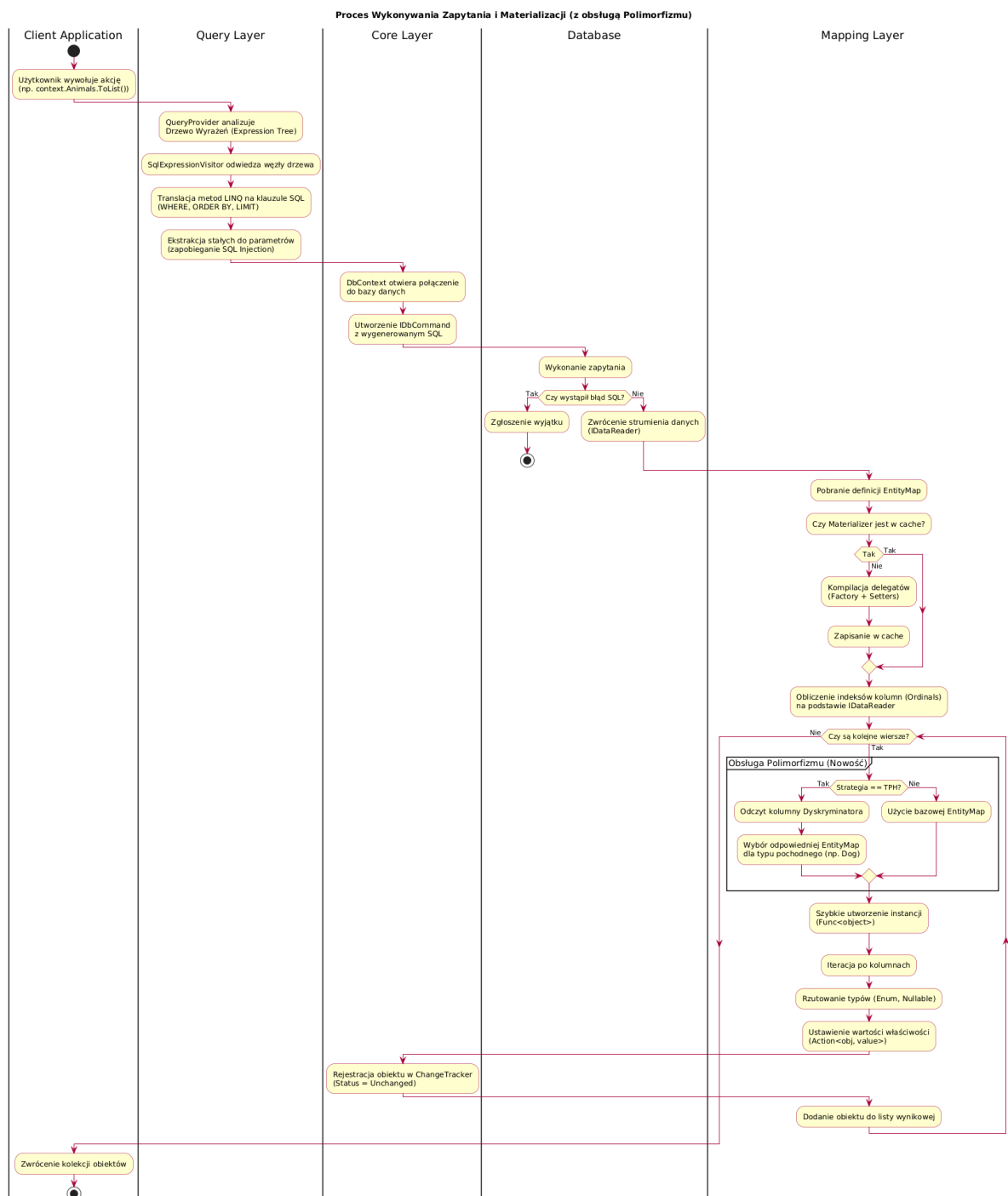


Figure 9: Diagram aktywności procesu przetwarzania zapytania.

33 Osiagniete cele

Zrealizowany projekt spełnia wszystkie założenia początkowe. Udało się stworzyć funkcjonalny system ORM, który:

1. Pozwala na mapowanie klas C# na tabele SQLite bez użycia plików XML.
2. Obsługuje operacje CRUD z zachowaniem transakcyjności (*Unit of Work*).
3. Udostępnia interfejs LINQ do budowania zapytań w sposób bezpieczny (*SQL Injection safe*).
4. Charakteryzuje się wysoką wydajnością dzięki unikalnej architekturze opartej na *Expression Trees* i cache'owaniu metadanych.

34 Napotkane wyzwania

Największym wyzwaniem technicznym okazała się implementacja `ObjectMaterializer` oraz `SqlExpressionVisitor`. Konieczność manualnego operowania na Drzewach Wyrażeń oraz dynamicznej kompilacji kodu (IL generation) wymagała głębokiego zrozumienia mechanizmów platformy .NET. Dodatkową trudnością było zapewnienie spójności typów między bazą danych (np. `INTEGER` w SQLite) a systemem typów C# (np. `bool`, `enum`, `Nullable<int>`).

35 Plany rozwoju

Projekt stanowi solidną bazę do dalszego rozwoju. Potencjalne kierunki rozbudowy to:

- **Snapshot Tracking:** Implementacja mechanizmu wykrywania zmian na poziomie poszczególnych właściwości, co pozwoliłoby na generowanie oszczędnych zapytań `UPDATE` (tylko zmienione kolumny).
- **Pełne wsparcie dla relacji:** Rozbudowa LINQ Providera o obsługę metod `Include()` oraz `Join()`, co pozwoliłoby na ładowanie całych grafów obiektów jednym zapytaniem.
- **Migracje:** Stworzenie narzędzia CLI do generowania skryptów aktualizujących schemat bazy danych na podstawie zmian w kodzie C#.